

Extension of a Simulation Software to Incorporate Quality-Related Factors in Investigations on Software Engineering Economics

David Kuhlen*

*IU University of Applied Sciences
Waterloohain 9, 22769 Hamburg, Germany
david.kuhlen@iu.org*

Andreas Speck

*Christian-Albrechts-Universität zu Kiel
Hermann-Rodewald-Straße 3, 24118 Kiel, Germany
aspe@informatik.uni-kiel.de*

Received 10 March 2022

Accepted 23 August 2022

Published 7 December 2022

The economics of software engineering [B. W. Boehm, *Software Engineering Economics*, Prentice-Hall, Englewood Cliffs, New Jersey, 1981] are strongly influenced by the way how software development processes are performed. Simulations give valuable insights in the performance of these processes. In order to research the economics of software engineering, a process analysis studio (PAS) was developed and presented [D. Kuhlen and A. Speck, Business process analysis by model checking, in *Proc. 5th Int. Symp. Data-Driven Process Discovery and Analysis SIMPDA*, eds. P. Ceravolo and S. Rinderle-Ma, 2015; D. Kuhlen and A. Speck, The way of designing a simulation software in order to evaluate the economic performance in software development, in *ICCMS '17 Proc. 8th Int. Conf. Computer Modeling and Simulation, Int. Conf. Proc. Series by ACM*, 2016; D. Kuhlen, *Kostensenkungspotenzialfunktion der Softwareproduktion*, dissertation, Christian-Albrechts-Universität zu Kiel, 2019]. This simulation software was built to analyze the capacitive configurations of the software production. It helps to assess the impact of different configurations on the process output. However, this research emphasizes the impact of quality-related factors (QRFs) on the process output. This paper presents a proposal how to extend the simulation software model in order to facilitate qualitative analysis. In order to facilitate the understanding of the model and the evaluation of the contribution, offered by the proposal, Prolog codes are used.

Keywords: Software engineering; software development; IT-management; quality-related factors; simulation.

*Corresponding author.

1. Introduction

As a consequence of competition in the field of software engineering economics (SEE), software producing companies (SPC) seek for optimized production procedures which have less costs [10, 11, 31]. Enhanced techniques for analyses facilitate business to improve the software production process (SPP). This paper focuses on the simulation-based analysis of software production processes (SASPP).

Simulations offer the potential to analyze the performance of SPPs [10, p. 2; 15, p. 14]. The software process has an increased complexity, because of multiple feedback loops [2, p. 6; 15]. Such processes can hardly be analyzed by other techniques [29]. Previous investigations in [21] or [18] showed a valid method that facilitates the analysis of SPPs by using a *process analysis studio* (PAS) software, first presented in [16]. However, the current version of the PAS is optimized to the analysis of configuration-related attributes of production processes. It helps to answer questions about how to plan processes, in order to reach advantages for the production costs. However, the performance of software development is also dictated by decisions, made during software development [10, 15, 33]. In order to evaluate the business impact of different management decisions, a changed simulation model is needed.

The current version of the PAS is heavyweight. It consists of multiple applications which have to be installed [21]. Current research revealed the software development process to be directed by the decisions of different characters [13]. In order to get further insights into the economic progression of software development, a new investigation is started. This research investigates different types of behavior of actors within the development [33]. The analysis of quality-related factors (QRFs) becomes important. This paper analyzes the current simulation model. On the basis of this analysis, a new proposal for an enhanced simulation model will be developed.

In this paper we want to clarify *how to design a simulation, giving further insights into the SEE, regarding to QRFs*. To contribute to the field of SEE, especially to SASPP, we want to answer the following questions:

- RQ1:** *How to describe cause and effect relations in the SEE? Which levels cover aspects, regarding to the SEE?*
- RQ2:** *How could a model of SPP be formalized, that could be completed with situational details, fitting to different investigations, in order to facilitate the analysis of the impact, QRFs have on the success of SEE?*
- RQ3:** *How to change the simulation software in order to facilitate dynamic investigations on the impact of QRFs?*

In order to answer these research questions, the paper presents the related work in the next section. The section ends by presenting cause and effect relationships (CaER) which have an impact on the economic success of SEE. On the basis of these

findings, Sec. 3 presents the research method. Prolog is used to describe a formalized model which helps to facilitate the understanding of the underlying model. A solution is proposed in Sec. 5. This proposal is evaluated in Sec. 6 on the basis of an experiment, executed in Prolog.

2. Related Work

Current research within the field of software engineering analyzes how management decisions affect the costs. Many different articles are related to the topic of this research. This paper addresses the fields of *simulation software* and *software engineering investigation*. Within the field of *simulation software* other articles like [20, 26, 28, 31] are interesting. Other articles, such as [27], analyze the dynamics within software development.

Within his work, Jørgensen analyzes the utilization of simple heuristics, used by management in order to make decisions [13]. Jørgensen compared current decision situations with the *time-saving-bias*. Based on two given teams which have different productivity he asked managers to decide which of the team's productivity should be improved, in order to decrease the total duration [13]. Jørgensen revealed that management tends to use simple heuristics which does not lead to correct decisions [13, p. 6]. On the basis of the evaluation of a survey, Jørgensen suggests management to be trained in making resource decisions [13, p. 6].

However, next to train management, there would be another way to improve the correctness of business decisions. Many kinds of decisions (like those of the analysis of Jørgensen) could be answered by performing simulations. An early model for simulations was described by Abdel-Hamid. The model of Abdel-Hamid gives management a framework to test their decisions [1, p. 19]. Abdel-Hamid's model bases on a system dynamics simulation. Such simulations were widely used, for example by van Oorschot *et al.* On the basis of the dynamics within software projects which are described in the literature, e.g. [3], van Oorschot *et al.* analyzed the effect of different constant sprint lengths in agile projects [24]. They used simulation techniques and revealed that the optimal length of iterations should be between 2 and 3 months [24, p. 24]. Recently, this model was used by Dong. Dong analyzes a way on how to simulate the software development in agile projects [7].

In order to facilitate the decision making, an above-mentioned process analysis studio (PAS) software was used in prior work, focusing on the productivity of SEE [16, 17]. The current simulation model of the PAS is mainly described in [18, 21]. The concept of the PAS allows the creation of processes. A process is defined as a sequence of flows [14, p. 439]. Matching to this, the PAS understands a process to be set of flows, each flow connecting two activities [16]. The simulation executes the behavior of actors [19]. Each activity (*=vertex*) is connected to actors on the basis of a team (called ActorGroup). The current version of the PAS works on the basis of a web application [21]. The web application will be used in order to create simulation models [21]. Users have the possibility to configure the processes by creating

activities, actors, organizations and so on [18]. The different steps which the PAS needs to perform analyses are described in Listing 3 in a simplified manner.

The PAS was created to execute simulations on multiple engines [21, p. 345 ff.]. After the model was created, another client solution will be executed to perform the simulations. Multiple clients are synchronized due to their algorithms, using the database [21]. Based on this, the workload of a simulation could be divided between these clients. This concept bases on the idea of creating orders for each simulation task. The illustration in Fig. 1 shows the structure of the PAS. It shows just a key part of the simulation model. The whole application kernel consists of more than 200 classes. Therefore, Fig. 1 is just giving an overview.

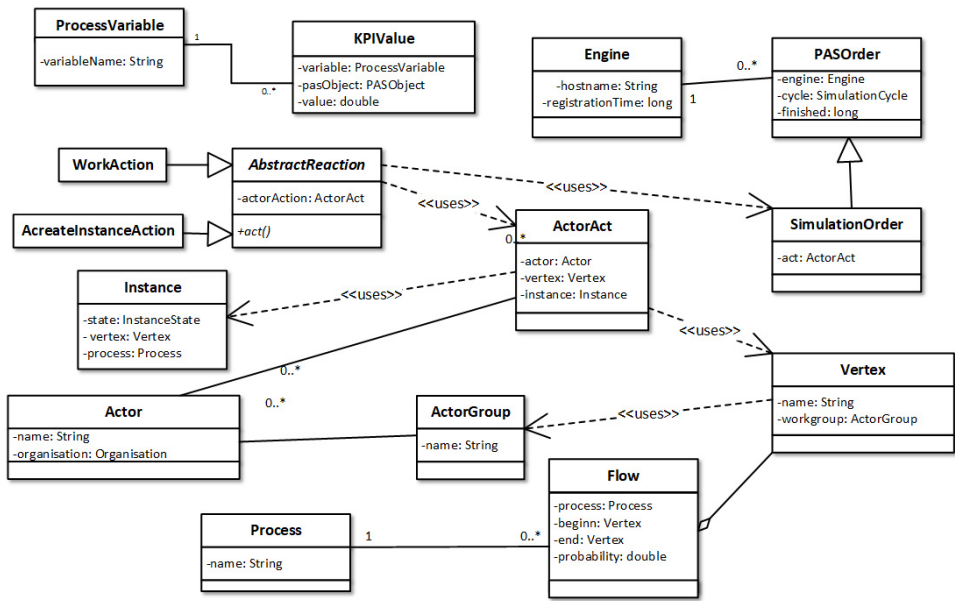


Fig. 1. Short view on the simulation model of the PAS, new illustration on the basis of [18], created with [23].

Next to other restrictions, the current version of the PAS is designed to analyze capacitive configurations of the SPP [21, p. 8]. Especially qualitative properties are hard to be analyzed by the current version of the PAS, because their analysis was excluded [21, p. 9, 413]. Due to this restriction, different qualitative ways how activities should be executed during the software development, could be hardly analyzed. However, as presented in [21, p. 413] enhancements of the simulation model could be sensible, in order to incorporate the analysis of QRFs.

This research bases on research in the field of SEE, especially regarding to the SPP and the field of simulation. It aims to contribute to improvements of the SPP. Elfaiz and Raharjo analyzed obstacles in the software process improvement [8]. On the basis of a systematic literature review, Elfaiz and Raharjo revealed 13

obstacles, like *time pressure*, *lack of knowledge transfer*, *lack of resources or budget*, *lack of roles and responsibility* etc. [8, p. 5]. Some of the found obstacles, like a *lack of trust*, are qualitative issues which are hard to analyze with the current version of the PAS, as they were excluded in previous versions.

Simulations are used by multiple studies, too, in order to analyze the performance of software development. In the recent past, a lot of research was published within the field of SEE which could be supported by an enhanced simulation model.

For example, the impact of management decisions within the SPP is an objective in the investigations of Kumar *et al.* [22]. Kumar *et al.* used simulation techniques in order to analyze the effect of task priorities on the throughput, within the software development. Considering bottlenecks within the process and different management strategies regarding to the preemption of prioritized tasks, they analyzed how changes in priorities affect the process output [22]. This shows that setting priorities has to be incorporated within the analysis of software development.

The use of agent-based simulations for open-source software development was analyzed by Zhao *et al.* [33]. Zhao *et al.* developed an agent-based model which simulates the behavior of software developers. The model was configured and tested on the basis of data which was collected from GitHub by Zhao *et al.* [33]. On the basis of GitHub, Zhao *et al.* distinguished different types of behavior [33]. The historical data from the repository was used to perform a regression analysis. This regression facilitated Zhao *et al.* to formulate the behavior of the agents. The resulting simulation model could serve as a prediction model. This model helps to predict the behavior of developers [33]. On the basis of this model, Zhao *et al.* could perform an experiment to analyze the impact of changing isolated parameters, such as the difficulty of tasks or the type of developers [33]. Such parameters are QRFs as well.

```
causes(increase(training), increase(skill)).
causes(decrease(skill), increase(duration)).
causes(increase(duration), increase(cost)).
causes(decrease(skill), decrease(cost)).
causes(increase(effort), increase(quality)).
causes(increase(effort), increase(duration)).
causes(increase(quality), increase(reusability)).
causes(increase(reusability), increase(flexibility)).
causes(increase(flexibility), increase(sale)).
causes(increase(sale), decrease(cost)).
causes(increase(policy), increase(reusability)).
causes(increase(requirement), increase(effort)).
causes(increase(requirement), increase(functionality)).
causes(increase(functionality), increase(sale)).
```

Listing 1. Description of cause and effect relations in SEE, formalized as Prolog-code.

The achievements from related work show cause and effect relationships, e.g. the impact of *knowledge* as thematized by [8] or the impact of the behavior of developers as discussed by [33]. Regarding to RQ1, we analyzed the cause and effect relations.

We assumed multiple cause and effect relations. These relations are illustrated in Listing 1 as Prolog-Code. The relation *cause* (A, B), having the parameters A and B , describes effects in the SEE. The relation *cause* defines the effect A to cause effect B . As for instance, the skill impacts on the costs and training impacts on the skill. Furthermore, Listing 1 describes how changes of the factors take affect such as an increase of sale could decrease costs (per unit) because software engineering mainly leads to overheads. These overheads come from the high amount of personal costs [6, p. 1055].

3. Analysis

These findings suggest to answer the research questions, especially RQ2, to facilitate further investigations on the performance of the SPP. Answering the research questions RQ2 and RQ3 requires the construction of a model. Software engineering uses models to describe the concept or architecture of a solution. The regular design process, being part of requirements engineering, is shortly described in Fig. 2.



Fig. 2. Regular design process in SEE, created with [Microsoft Visio Plan 2].

In order to contribute to the state of the art in science, we use an approved research methodology. The method needs to fit to the regular process which is necessary to design new solution models. In order to develop a new simulation model, the *design science* research methodology, described e.g. by Hevner *et al.* [12] and Peffers *et al.* [25] will be used.

Referring to the guidelines for design science research, presented by Hevner, March and Park, the next section describes the “*Problem Relevance*” [12, pp. 84–85]. This section will describe the problem, the motivation and the requirements for a new solution [25, p. 46]. In order to describe the problem, the findings form literature review are used. Section 5 presents the proposed model, referring to “*Design as an Artifact*” [12, pp. 82–84]. The enhancement in the simulation model is presented on the basis of UML. This section demonstrates the conceptualized model. The artifact becomes evaluated in Sec. 6, referring to “*Design Evaluation*” [12, pp. 85–87]. In Sec. 7, the “*Research Contributions*” [12, p. 87] which were obtained through the whole process are summarized.

In order to fulfil Hevner, March and Park’s “*Guideline 5: Research Rigor*” [12, pp. 87–88], we use Prolog. The logical programming language facilitates the design of a model in a formalized way and its use in executions. This formalization supports the understanding of the model and its relation. The art of programming in Prolog seems to be understandable “both to technology-oriented as well as management-oriented audiences”, as required in “*Guideline 7: Communication of Research*” [12, p. 83]. Furthermore, as part of the evaluation, we formulate relations in Prolog, assuming these relations should formalize the required research interest of

RQ2 and RQ3. Prolog helps to evaluate whether the designed solution will be able to fulfill these research questions or not. As RQ1 is part of the preparatory work and because of its nature, our proposed answer to this question will be excluded from evaluation. In order to perform the evaluation, SWI-Prolog will be used, see [30].

4. Requirement

The previous explanations showed the current version of the PAS to be optimized for some investigation purposes. As the analysis of QRFs was excluded from the PAS, an enhancement is required because we expect QRFs to influence the success of SEE.

Regarding to RQ1, the literature review suggests aspects which take an influence on the software development and could be divided into four dimensions, as displayed in Fig. 3. Next to quantitative aspects [4] [10, p. 2] [20, p. 322], qualitative aspects [2] [4] [27] [31, p. 1 & p. 28] also influence the success of development projects. Furthermore, the success of software development is influenced by the SPP itself [4, 9, 10]. However, there are more existing factors which could not be matched to the process. Such factors within the *situation related dimension* influence the software development but are not covered by the process itself [27].

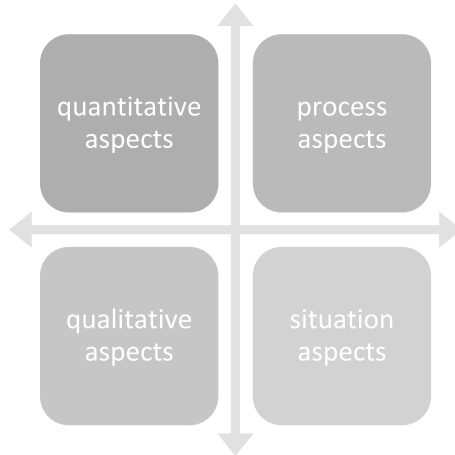


Fig. 3. Proposed model, classifying aspects which influence the SEE, created with Microsoft Word as part of Microsoft Office Home and Business 2019 which is a product of the Microsoft Corporation.

The analysis of different management decisions will be possible if alternative situations and possible decisions, matching these situations, could be expressed within a model. A situation which requires a management decision is given by the state of different objects (e.g. the number of tasks, the workload of the team, the similarity of the current task to the functionality of the existing software...).

During the simulation, this state is described as *simulation_model*, covering the *process_model* and the key performance indicators (KPIs) which describe the current situation. Most of these attitudes could be expressed as values of key performance indicators. Important management decisions aim to *handle* the cause and effect relations, described in Listing 1. By simplifying the possible alternatives, a decision could aim to *increase*, *decrease* or *stabilize* factors within the cause-and-effect relationship (CaER), as described in Listing 2.

```
handle(Factor, X):- X = increase(Factor).
handle(Factor, X):- X = decrease(Factor).
handle(Factor, X):- X = stable(Factor).
```

Listing 2. Handling of CaER-factors, formalized as Prolog-code.

Decisions consist on the situation (circumstances of the environment) and their impact. Next to the *KPIs* which describe a situation a management decision has an impact on the progression of the software development.

The required model changes to incorporate QRFs in the analysis need to fulfil the following requirements:

- Referring to RQ2: A ‘quality-functor’ has to be designed which influences the economic situation during the progression. This functor is a mapping of the inputs quality, the simulation model and the case to the outputs case and the simulation model. The case describes the object being processed during the process while it is executed in the simulation. The quality should be defined for individual entity types with individual levels of details. $qFunctor(quality, simulation_model, case) \rightarrow (case, simulation_model)$.
- Referring to RQ3: The quality should be influenced by itself, in order to facilitate dynamic investigations.

5. Concept for a New Simulation Solution

In order to analyze the impact of QRFs, the PAS has to be changed. To lead to an improvement, this section comes with a proposal for an improvement of the PAS. This improvement aims at the progression within the simulation. In order to describe the change adequately, important parts of the current version of the PAS have to be described.

The current version of the PAS works on the basis of a web application. The web application will be used in order to create process models. Users have the possibility to configure the processes by creating activities, actors, organizations and so on [18]. After the creation of the model, the process analysis studio engine (PASE) will be executed to perform the simulations. Multiple clients are synchronized due to their algorithms, using the database.

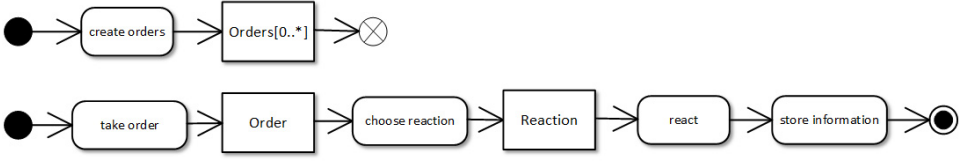


Fig. 4. Current Process, describing the progression of the PAS, created with [Microsoft Visio Plan 2].

```

:- dynamic perform/3.

perform(createOrganisation, [], [organisation]).
perform(defineSalary, [], [salary]).
perform(createActor, [organisation], [actor]).
perform(configureActorKPIs, [salary],
         [actor, actor_kpis]).
perform(defineRisks, [], [risks]).
perform(createActorGroup, [actor],
         [actorGroup, group_size]).
perform(configureActorGroupKPI, [group_size],
         [actorGroup_kpi]).
perform(createActivity, [actorGroup, effort],
         [actorGroup, activity]).
perform(defineEffort, [], [effort]).
perform(createFlow, [activity], [activity, flow]).
perform(configureFlow, [risks], [flow_kpis]).
perform(createProcess, [flow], [process]).
perform(buildProcessModel, [actor, activity, flow,
                           process], [process_model]).
perform(buildKpiModel, [actorGroup_kpi, actor_kpis,
                       flow_kpis], [kpi_model]).
perform(buildSimulationModel,
         [kpi_model, process_model], [simulation_model]).
perform(modelQuality,
         [actor, instance, activity], [quality]).
perform(createInstanceAction, [actor, activity],
         [instance, createInstanceOrder]).
perform(moveAction, [instance, simulation_model],
         [instance_cycle, moveInstanceOrder]).
perform(workAction, [instance, simulation_model],
         [instance_duration, workOrder]).
perform(executeSimulation, [createInstanceOrder,
                             moveInstanceOrder, workOrder], [progression]).
perform(costAnalysis, [instance_cycle,
                      instance_duration, simulation_model,
                      progression], [cost]).
  
```

Listing 3. Description of the current simulation behavior of the PAS as Prolog-code.

Figure 4 displays the two main processes, the PASEs are following. The execution bases on orders, being created. An order describes an action which is executed by an actor. Different kinds of orders are executed in a special order, regarding to the process execution. This model facilitates the analysis of configuration-related aspects. In order to perform SASPPs, multiple objects have to be created in a special order. By Listing 3 important steps of the simulation are described in a simplified manner. The relation *perform* (S, I, O) describes those steps. Each step (S) requires a list of inputs (I) and results in a list of outputs (O). Furthermore, Listing 3 formalizes the order of necessary steps which has to be executed to perform the SASPP.

The main part which is responsible for the qualitative, semantic behaviour of actors is the reaction. Currently, actors could react in four ways: they could *create instances* (=like tasks), they could *work on the instances, tasks*, they could *move the instances within the process* and they could *be waiting for work* [18, 21]. Those types of reactions are generic, regarding the processing. As it is described in Fig. 1, reactions are derived from the abstract class *AbstractReaction* [18, 21]. Obviously, this model facilitates the expansion in order to introduce more kinds of reactions. However, in Fig. 4 the step *react* which executes the reaction, aims at processing of the workload within the process model. Therefore, it would be an easy solution to add new subtypes of reactions.

However, this generic way of how to work, following a process, should not be changed. Another change-point is needed to make sure QRFs impact on the progression. To realize the introduction of qualitative analysis, we enhanced the model of the PAS, as described in Fig. 5.

In the new version of the PAS, the step *react* is embedded within two signals. These signals trigger a pre- and post-qualitative-reactions. This qualitative reaction could be defined individually, by using the interface *IPerforming* (see Fig. 6). The steps collect associated objects and perform their behaviour for each object, described in steps, too. A cascade of calls to the pre- and post-qualitative reactions follows. This allows qualitative behaviour to influence the processing without changing the processing itself.

The progression of the extension of the PAS could also be described on the basis of the *perform*-relation, showed in Listing 3. The change of the PAS to incorporate the analysis of QRF requires an addition of the two steps *preQAction* and *postQAction*, covering the signals *before reaction* and *after reaction* illustrated in Fig. 5. These steps have to be incorporated in the regular execution. Therefore, just the step “*executeSimulation*” has to be adapted.

Listing 4 describes the changes in the PAS in a simplified form, formalized in Prolog. The new version of the step “*executeSimulation*”, displayed in Listing 4, integrated the new steps, realizing an execution before- and after-reactions. In order to realize the new behaviour, described in Fig. 5, new classes are necessary. Figure 6 shows the enhanced class model. As illustrated, the *IPerforming* interface is implemented e.g. by the classes *Actor*, *Instance*, *Vertex* and so on. In investigations without the need to analyse the impact of QRF, a basis implementation of these

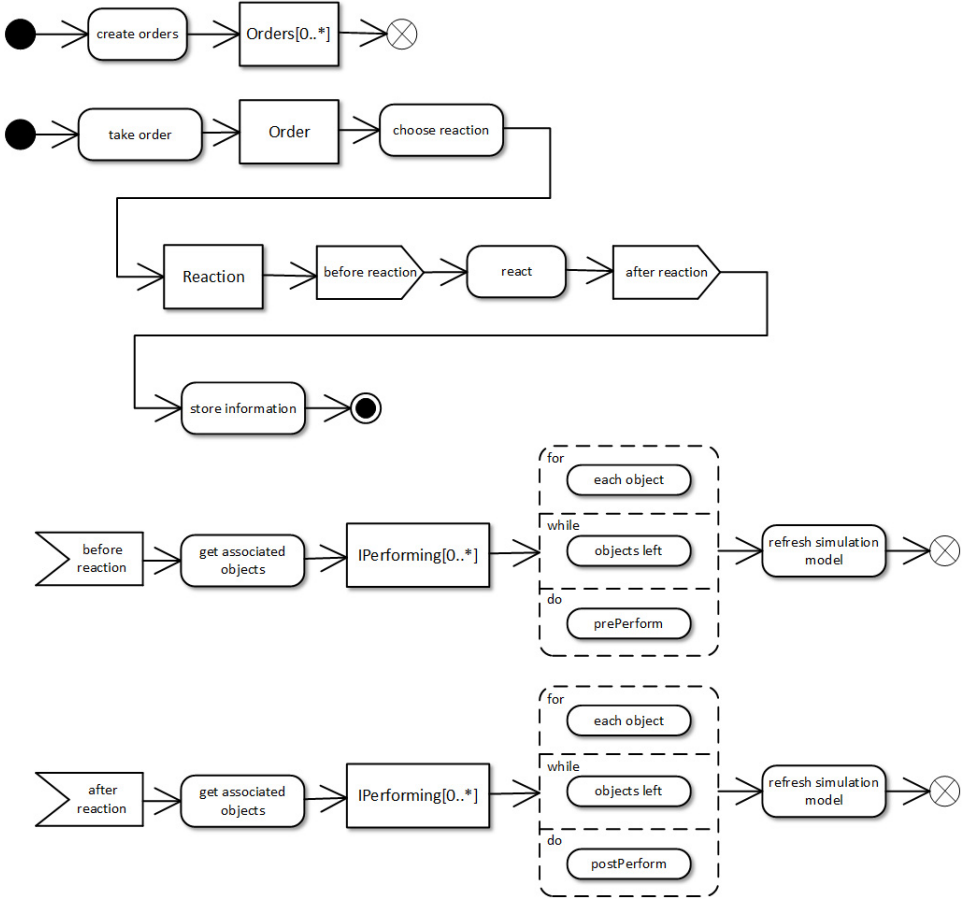


Fig. 5. New Progression Model, created with [Microsoft Visio Plan 2].

classes could be used. If a special behaviour is necessary, individual sub classes of these classes could be derived, offering a specialized qualitative behaviour. The use of such an interface serves as a generic solution. It facilitates the

```

perform(executeSimulation, [createInstanceOrder,
    moveInstanceOrder, workOrder, preQActionOrder,
    postQActionOrder], [progression]).
perform(preQAction, [simulation_model, quality,
    instance], [instance, simulation_model,
    preQActionOrder]).
perform(postQAction, [simulation_model, quality],
    [quality, simulation_model, postQActionOrder]).
  
```

Listing 4. Adapted and added steps of the PAS, description simplified as Prolog-code.

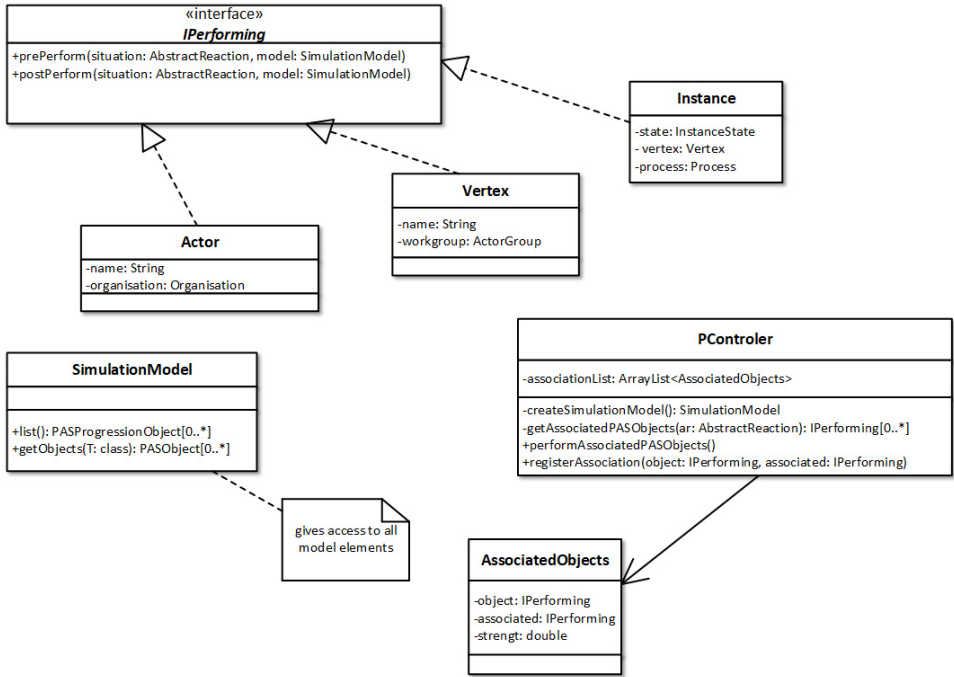


Fig. 6. Enhanced class model of the PAS, created with [Microsoft Visio Plan 2].

implementation of own types, describing individual behaviour. This provides a solution, regarding to RQ2.

The execution of the new progression model, illustrated in Fig. 5, requires a new controller class *PController*. The controller is responsible for the collection of associated *PASObjects*. The collection bases on a register of associated objects. Next to this register, all *IPerforming* objects which are closely related to an instance will be collected to find associates.

The division of the steps *pre-* and *post-reactions*, which incorporates QRFs in the simulation, facilitates the implementation of dynamic simulation behaviour. Dynamic simulation capabilities require a factor influencing itself due to multiple feedback loops, leading to continuous modelling [15, 32]. Thus, the proposed enhancement of the PAS allows the incorporation of dynamic capabilities if the *post-QAction* impacts on the quality itself.

Figure 7 shows the interaction of *pre-* and *postQActions*. As presented in Fig. 6, the particular behaviour of these -QActions could be defined individually by adaptations in the source code, fitting to different investigations. In order to fulfil the requirement of RQ2, the proposed model defines an interface *IPerforming* which could be implemented on an individual basis for different investigation purposes. This implementation should be designed to facilitate dynamic investigations

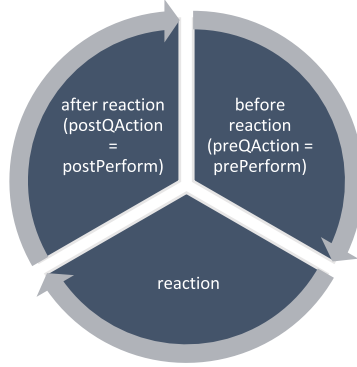


Fig. 7. Interaction of pre- and postQAction reactions, created with Microsoft Word as part of Microsoft Office Home and Business 2019 which is a product of the Microsoft Corporation.

(see RQ3). Therefore, it would be recommended to use the *preQAction* to make sure the quality impacts on the success. The *postQAction* could be used in order to make sure that the model of quality itself changes. This could lead to dynamic behaviour.

6. Analysis of the Sustainability

This analysis aims at demonstrating the benefit of the proposed extension. It bases on logical conclusions on the basis of the assumed relations. In order to achieve some formality and objectivity regarding to the evaluation of the logical conclusions [30], will be used. However, the assumptions and the assumed logical relations cause the evaluation to be subjective, partially. In order to decrease subjectivity, the assumptions and the assumed logical relations could be adjusted, suitable to the respective circumstances, e.g. in corporate practice, development projects or research projects.

First, it has to be analyzed whether it is meaningful to incorporate QRFs in economic analyses. This first evaluation bases on the CaERs are described in Listing 1.

```

impact(Trigger, Effect):-
    causes(T, E),
    handle(Trigger, T),
    handle(Effect, E).

analysis_meaningful(FactorToBeChanged, Target) :-
    impact(FactorToBeChanged, Target).
analysis_meaningful(FactorToBeChanged, Target) :-
    impact(FactorToBeChanged, X),
    analysis_meaningful(X, Target).

```

Listing 5. Analyze the impact of QRFs, formalized as Prolog-code.

In order to evaluate whether it is meaningful to incorporate QRFs in simulation analyses, it has to be established if QRF impacts on the economic success of SEE. The incorporation of QRFs in SASPP could be seen as sensible if the QRF has a direct or transitive impact. This is described by the relation *analysis_meaningful* in Listing 5. As the costs are relevant for the success of SEE, we could perform the query illustrated in Listing 6. The query would succeed with a positive result, using SWI-Prolog version 8.2.4.

```
?- analysis_meaningful(quality, cost).
true .
```

Listing 6. Relevance of QRF, executed in SWI-Prolog version 8.2.4.

The next step in the evaluation aims to determine whether the change of the PAS, described in Listing 4, facilitates the analysis of QRFs. Therefore, a relation *possibleImpactAnalysis(A, B)* is required. This relation should confirm if factor A influences factor B within the simulation. In order to determine this influence, a relation *toDo* is necessary which determines the steps (*S*) being necessary to influence factor *B*. The execution of those steps (*S*) leads to (output-)data (*D*). If this output (*D*) contains the factor *A*, the factor *A* impacts on *B*. The whole evaluation is presented in Listing 7.

Obviously, the relations *contains(List, Element)* and *containsAll(MainList, SubList)* are well known functors, confirming whether an element or a sublist is part of a list. The query `?- containsAll([a, b, c], [a, b]).` leads to the answer **true** in Prolog interpreters, e.g. SWI-Prolog version 8.2.4. Clockin and Mellish described similar relations *member* and *subset* [5, p. 160]. The relation *append* allows to connect lists [5, p. 63]. In order to simplify the illustrations a relation *echo(_)* `:- write('——-').` is used, showing 'headlines' in the terminal.

On the basis of the relations displayed in Listing 7, it could be evaluated whether it is possible to analyze the impact of QRF on the economic success of software production after the model of the PAS will be changed. If the enhancement of the model facilitates the analysis of QRFs, it should be possible to analyze the impact of quality on the success. In terms of the simplified Prolog-code, the query `?-possibleImpactAnalysis(quality, cost)` should succeed after the change of Listing 4 was implemented.

In order to perform the evaluation, regarding to RQ2 and RQ3, an experiment will be accomplished on the basis of the relations, displayed in Listing 7 and the above-mentioned relations. The model of the current version of the PAS (5.1.16) which is simplified in Listing 3 is contained in the file *pas_simplified_model.Version5116*.

The proposed changes which are modeled in Listing 4, in a simplified manner, should affect the current model. Therefore, the old step *executeSimulation* has to be

```

possibleImpactAnalysis(InfluenceOf, InfluenceOn) :-
    toDo(InfluenceOn, ListNecessarySteps),!,
    collectedInputs(ListNecessarySteps,
                    ListCollectedInputs),
    contains(ListCollectedInputs, InfluenceOf).

toDo(TargetObject, ListNecessarySteps) :-
    perform(Step, [], ListOutputs),
    contains(ListOutputs, TargetObject),
    append([], [Step], ListNecessarySteps).
toDo(TargetObject, ListNecessarySteps) :-
    perform(Step, InputsNeeded, ListOutputs),
    contains(ListOutputs, TargetObject),
    toDoForObjects(InputsNeeded,
                    NecessaryStepsForInput),
    append(NecessaryStepsForInput, [Step],
           ListNecessarySteps).

toDoForObjects([], []).
toDoForObjects(
    [FirstTargetObject|RemainingListOfTargetObjects],
    ListOfNecessarySteps) :-
    toDoForObjects(RemainingListOfTargetObjects,
                    RemainingListOfNecessarySteps),
    toDo(FirstTargetObject, StepsForTargetObject),
    append(RemainingListOfNecessarySteps,
           StepsForTargetObject, ListOfNecessarySteps).

collectedInputs([], []).
collectedInputs(
    [FirstPerformedStep|ListOtherPerformedSteps],
    ListCollectedInputs) :-
    collectedInputs(ListOtherPerformedSteps, L),
    perform(FirstPerformedStep, _, Outputs),
    append(Outputs, L, ListCollectedInputs).

```

Listing 7. Necessary relations to evaluate the possible impact of QRFs in Prolog.

replaced. In order to replace the predicate, it was removed by using the built-in predicate *retract* [5, p. 127] at first. The new version of the step *executeSimulation* was inserted by using the built-in-predicate *assertz* [5, p. 126] which will be used to add the steps *preQAction* and *postQAction*, too.

First, Listing 8 shows that the current version of the PAS facilitates the analysis of effort, impacting the economic success of software engineering. However, the current model does not support the analysis of quality's impact on the economic success. The query *?- possibleImpactAnalysis(quality, cost).* does not succeed until the simulation model will be changed. After the simplified model of the PAS was changed as proposed, the ability of the simulation software changes. The

```

?- consult(pas_simplified_model_Version5116).
true.

?- echo('Evaluate the ability of the current model to analyze
QRFs...').
-----
true.

?- possibleImpactAnalysis(effort, cost).
true .

?- possibleImpactAnalysis(quality, cost).
false.

?- echo('Now the simulation model will be changed for
evaluation...').
-----
true.

?- retract(perform(executeSimulation, [createInstanceOrder,
moveInstanceOrder, workOrder], [progression])).
true.

?- assertz(perform(executeSimulation, [createInstanceOrder,
moveInstanceOrder, workOrder, preQActionOrder,
postQActionOrder], [progression])).
true.

?- assertz(perform(preQAction, [simulation_model, quality,
instance], [instance, simulation_model, preQActionOrder])).
true.

?- assertz(perform(postQAction, [simulation_model, quality],
[quality, simulation_model, postQActionOrder])).
true.

?- echo('Evaluate the ability of the proposed model to
analyze QRFs...').
-----
true.

?- possibleImpactAnalysis(effort, cost).
true .

?- possibleImpactAnalysis(quality, cost).
true .

```

Listing 8. Evaluation of the proposed extension for the PAS simulation model, executed in SWI Prolog version 8.2.4.

```

hasDynamicImpact(DynamicImpactFactor, InfluenceOn) :-
    perform(StepA, ListInputsStepA, _),
    contains(ListInputsStepA, DynamicImpactFactor),
    perform(StepB, _, ListOutputsStepB),
    contains(ListOutputsStepB, DynamicImpactFactor),
    toDo(InfluenceOn, ListNecessarySteps),
    containsAll(ListNecessarySteps, [StepA, StepB]).

```

Listing 9. Relation to determine whether the model has dynamic impact in Prolog.


```

?- consult(pas_simplified_model_Version5116).
true.

?- echo('Evaluate the ability of the current model to
facilitate dynamic investigations of QRF...').
-----
true.

?- hasDynamicImpact(quality, cost).
false.

?- echo('Now the simulation model will be changed in two
steps for evaluation...').
-----
true.

?- retract(perform(executeSimulation, [createInstanceOrder,
moveInstanceOrder, workOrder], [progression])).
true.

?- assertz(perform(executeSimulation, [createInstanceOrder,
moveInstanceOrder, workOrder, preQActionOrder,
postQActionOrder], [progression])).
true.

?- assertz(perform(preQAction, [simulation_model, quality,
instance], [instance, simulation_model, preQActionOrder])).
true.

?- echo('Step 1: Are dynamic investigations possible if step
"postAction" is missing? Beginn analysis...').
-----
true.

?- hasDynamicImpact(quality, cost).
false.

?- echo('Result Step 1: Dynamic investigations of QRFs are
not possible if the step "postAction" is missing.').
-----
true.

?- assertz(perform(postQAction, [simulation_model, quality],
[quality, simulation_model, postQActionOrder])).
true.

?- echo('Step 2: Are dynamic investigations possible after
the step "postQAction" has been added? Beginn analysis...').
-----
true.

?- hasDynamicImpact(quality, cost).
true .

?- echo('Result Step 2: Dynamic investigations of QRFs are
possible if the step "postAction" is added.').
-----
true.

```

Listing 10. Result of the evaluation of the contribution of the proposal to RQ3, executed Prolog code in SWI-Prolog, version 8.2.4.

Regarding to the evaluation of RQ3, a relation has to be formulated which determines whether the proposed model facilitates dynamic investigations. In order to evaluate the contribution of the proposed enhancement of the PAS to RQ3, the relation *hasDynamicImpact*(*Df*, *Io*) was defined as illustrated in Listing 9. This relation confirms if a factor (*Df*) has a dynamic impact on another factor (*Io*).

As illustrated in Listing 10, the evaluation revealed dynamic investigations to be facilitated by the proposed addition of the step *postQAction*. This confirms the proposed model to be a valid solution, fulfilling the requirement of RQ3.

The analysis of software production processes via simulations gives helpful insights in software engineering economics (SEE) [4]. Factors which influence the productivity should be incorporated in the simulation to facilitate the analysis of different decision alternatives. This could help decision makers to perform well. Next to capacity-oriented factors, like staffing etc., quality-related factors (QRF) are also seen as an influence on the outcome of software production.

The execution of the behavior surrounds the regular processing. The processing which aims at the abundance of the process description should be surrounded by a pre-behavior and a post-behavior of performable objects. This leads to a division of processing and performing which also facilitates aspects of the software-maintenance. Furthermore, the division of pre- and post-behaviors could be used to realize dynamic investigations.

The results of this investigation are especially related to the PAS, presented in previous work, e.g. [16, 18, 21]. However, the proposed way of how to incorporate qualitative behavior in process simulations, could also be interesting for other types of simulations. After the evaluation in this paper showed the proposed change to be useful, the PAS has to be changed. After this change will be implemented, it could be interesting to investigate its impact on the performance during the simulation. Further research is necessary to evaluate the effectiveness of this proposal in other software solutions.

References

1. T. K. Abdel-Hamid, The dynamics of software development project management: an integrative system dynamics perspective, dissertation, Massachusetts Institute of Technology (1984).
2. O. Armbrust, T. Berlage, T. Hanne, P. Lang, J. Münch, H. Neu, S. Nickel, I. Rus, A. Sarishvili, S. van Stockum and A. Wirsén, Simulation-based software process modeling and evaluation, in *Handbook of Software Engineering and Knowledge Engineering: Recent Advances*, ed. S. K. Chang, Vol. 3 (World Scientific Publishing Company, Singapore, 2005), pp. 333–364. DOI: 10.1142/9789812775245_0012.
3. T. K. Abdel-Hamid and S. E. Madnick, *Software Project Dynamics: An Integrated Approach* (Prentice-Hall, New York, 1991). Pearson Education. ISBN: 0-13-822040-9.
4. B. W. Boehm, *Software Engineering Economics* (Prentice-Hall, Englewood Cliffs, New Jersey, 1981) ISBN 0-13-822122-7.
5. W. F. Clocksin and C. S. Mellish, *Programming in Prolog*, 5th edn. (Springer, New York, 2003). DOI: 10.1007/978-3-642-55481-0.
6. N. T. Danh, Potential challenges for software engineering applications and practical solutions, *J. Positiv. School Psychol.* **6**(8) (2022) 1051–1056.
7. W. Dong, Using a system dynamics simulation model to explore the validity of dynamics of Agile software development, Master thesis, Iowa State University, Iowa (2021). DOI: 10.31274/etd-20210609-47.
8. E. A. Elfaiz and T. Raharjo, The obstacles of software process improvement in software house: A systematic literature review and empirical study, *J. Phys. Conf. Ser.* **1811** (2021) 012102. IOP Publishing. DOI: 10.1088/1742-6596/1811/1/012102.
9. B. B. N. França and G. H. Travassos, Are we prepared for simulation based studies in software engineering yet? *CLEI Electron. J.* **16** (2013) 9. Centro Latinoamericano de Estudios en Informática, Available online on the website from SciELO Uruguay, Scientific Electronic Library Online at URL: www.scielo.edu.uy/scielo.php?pid=S0717-50002013000100009&script=sci_arttext, accessed 2017-OCTOBER-16. [Online-Version, ISSN 0717-5000].
10. J. A. García-García, J. G. Enríquez, M. Ruiz, C. Arévalo and A. Jiménez-Ramírez, Software process simulation modeling: Systematic literature review, *Comput. Stand. Interf.* **70** (2020) 103425. Elsevier. DOI: 10.1016/j.csi.2020.103425.
11. J. Heimicke, K. Dühr, M. Krüger, G.-L. Ng and A. Albers, A framework for generating agile methods for product development, in *31st CIRP Design Conference 2021 (CIRP Design 2021)*, ScienceDirect, Procedia CIRP 100, Elsevier Ltd., 2021. DOI: 10.1016/j.procir.2021.05.043.
12. R. Hevner, S. T. March and J. Park, Design science in information systems research, *MIS Quart.* **28** (2004) 75–105. DOI: 10.2307/25148625.
13. M. Jørgensen, Team resource management decisions in software development projects, in *2021 IEEE 23rd Conference on Business Informatics (CBI)*, 2021. DOI: 10.1109/CBI52690.2021.00011.
14. S. Kiebusch, B. Franczyk and A. Speck, An unadjusted size measurement of embedded software system families and its validation, *Soft. Process: Improv. Pract.* **4**(11) (2006) 435–446. Wiley Online Library.
15. M. T. Khatun, K. Hiekata, Y. Takahashi and I. Okada, Design and management of software development projects under rework uncertainty: A study using system dynamics, *J. Decis. Syst.* (Informa UK Limited, Taylor & Francis Group, 2022). DOI: 10.1080/12460125.2021.2023257.

16. D. Kuhlen and A. Speck, Business process analysis by model checking, in *Proc. 5th Int. Symp. Data-Driven Process Discovery and Analysis SIMPDA*, eds. P. Ceravolo and S. Rinderle-Ma, 2015, pp. 154–170.
17. D. Kuhlen and A. Speck, Project-economy, in *Meißner Hochschulschriften*, ed. F. Lang, 6th edn. (Hochschule Meißen (FH) und Fortbildungszentrum, Der Rektor Freistaat Sachsen, Germany, 2016). ISSN (Print) 2513-0579, ISSN (Online) 2513-0951.
18. D. Kuhlen and A. Speck, The way of designing a simulation software in order to evaluate the economic performance in software development, in *ICCMS '17 Proc. 8th Int. Conf. Computer Modeling and Simulation, Int. Conf. Proc. Series by ACM*, 2016. DOI 10.1145/3036331.3036347.
19. D. Kuhlen and A. Speck, Economic output under the conditions of social freedom in software development, *Int. Softw. Eng. Appl.* **7**(6) (2016) 17–31. DOI: 10.5121/ijsea.2016.7602.
20. O. S. Kristiansen, U. Sandberg, C. Hansen, M. Skovgaard Jensen, J. Friederich and S. Lazarova-Molnar, Experimental comparison of open source discrete-event simulation frameworks, in *Simulation Tools and Techniques, 13th EAI Int. Conf. SIMUtools 2021 Virtual Event, Proceedings*, eds. D. Jiang and H. Song, Cham, Switzerland: Springer Nature Switzerland AG, 2021, pp. 315–330.
21. D. Kuhlen, Kostensenkungspotenzialfunktion der Softwareproduktion, dissertation, Christian-Albrechts-Universität zu Kiel (2019).
22. M. Kumar, O. S. Vaidya and R. K. Srivastava, Impact of task priority on software supply chain: A simulation approach, *South Asian J. Bus. Stud.* **10**(3) (2021) 326–341. Emerald Publishing Limited. [DOI: 10.1108/SAJBS-03-2020-0069].
23. [Microsoft Visio Plan 2] Microsoft Corporation (year unknown): “Microsoft® Visio® Plan 2 MSO”. Version 2202.
24. K. E. van Oorschot, K. Sengupta and L. N. van Wassenhove, Dynamics of agile software development, in *Proc. 27th Int. Conf. System Dynamics Society*, Vol. 6, 2009, pp. 3910–3939. ISBN: 978-1-61567-607-1.
25. K. Peffers, T. Tuunanen, M. A. Rothenberger and S. Chatterjee, A design science research methodology for information systems research, *J. Manag. Inf. Syst.* **24**(3) (2008) 45–77. M. E. Sharpe, Inc. DOI: 10.2753/MIS0742-1222240302.
26. N. Rahman, A simulation model for application development in data warehouses, *Int. J. Oper. Res. Inf. Syst.* **9**(1) (2018) 66–80. IGI Global. DOI: 10.4018/IJORIS.2018010104.
27. L. M. Restrepo-Tamayo, G. P. Gasca-Hurtado, L. E. Machuca-Villegas and S. Morillo-Puente, Productivity of Software Development Teams Based on Social and Human Factors: A Dynamic Conceptual Model. In *Information Systems and Technologies*, eds. A. Rocha, H. Adeli, G. Dzemyda and F. Moreira, WorldCIST 2022. Lecture Notes in Networks and Systems, Vol 470 (Springer, Cham, 2022), https://doi.org/10.1007/978-3-031-04829-6_2.
28. A. Radecki and T. Rybicki, Simulation oriented layer of embedded software architecture for rapid development of custom embedded systems virtual simulators used in didactics, *Appl. Sci.* **12** (2022) 6322. MDPI. DOI: 10.3390/app12136322.
29. A. Speck, E. Pulvermüller and D. Heuzeroth, Validation of business process models, in *Correctness of Modelbased Software Composition, Proceedings ECOOP 2003 Workshop #11, 17th European Conference on Object-Oriented Programming, Technical Report No. 2003-13, Interner Bericht (Internal Report)*, eds. R. Van Der Straeten, A. Speck, E. Pulvermüller, M. Clauß and A. Pleuss (2003). Universität Karlsruhe Fakultät für Informatik. DOI: 10.5445/IR/3142003.
30. SWI Prolog “SWI-Prolog”. Version 8.2.4. <https://www.swi-prolog.org/download/stable>, accessed 2021-OCT-22 (2021).

31. J. O. Wesch, An implementation strategy for Tecnomatix Plant Simulation software, Master thesis, North-West University NWU(r) (2022). Available online on the website from North-West University at URL: <https://repository.nwu.ac.za/bitstream/handle/10394/39342/Wesch%20JO%20final%20copy.pdf?sequence=1>, accessed 2022-AUG-22.
32. M. Wu and H. Yan, Simulation in software engineering with system dynamics: A case study, *J. Softw.* **10**(4) (2009) 1127–1135. Academy Publisher. DOI: 10.4304/jsw.4.10.1127-1135.
33. Zhao, J. Cao, S. Jiang and Q. Qi, An agent based simulation system for open source software development, in *2020 IEEE World Congress on Services*, 2020, pp. 164–170. DOI: 10.1109/SERVICES48979.2020.00044.